

The pi Agent-Loop Fleet-30 Benchmark

A tool-use coding evaluation across the free / self-hosted model fleet

2026-07-06 (report v7)

1 Why this benchmark exists

The models we run day-to-day are not chatbots — they are the reasoning core of *agent loops*: they read a task, call tools (write files, run shells, inspect output), observe the result, and iterate until a verifier says the job is done. A model can be fluent in isolation and still be useless in a loop if it cannot follow a file-path discipline, recover from a failing test, or drive a multi-step tool sequence without wandering. Standard leaderboards (MMLU, GSM8K, chat-arena Elo) measure none of this.

Fleet-30 measures exactly this: the ability of a model to *close an agent loop on a real coding task*. Every problem is run through the `pi` agent CLI (Earendil Pi) with **tools ON**, against a deterministic verifier that returns a hard exit code — never model prose. The verdict is binary per problem: the loop either produced code that passes an independent grader, or it did not.

The benchmark serves three concrete goals for the fleet:

- **Enablement mapping.** We serve dozens of models across local DGX endpoints, nVIDIA A100 GPUs, Intel PVC tiles, spark Ollama nodes, and OpenRouter. This tells us *which of them can actually drive an agent loop*, so we route real agentic work (Kanban workers, coordinators, code review) to models that clear the bar and keep the rest for single-shot generation.
- **Serving-path validation.** The same model on two backends is a different product. A broken chat template, a missing tool-parser, a quantization, or a streaming-flush bug in the server all change the score. Fleet-30 is run *per serving path*, so a lane is a (model $\times$ backend) pair, and a regression in the stack shows up as a score drop.
- **A shared coding curriculum.** The 30 problems are an authored progression — write-from-spec, fix-failing-tests, optimize-to-target, best-of- N tournaments, and critic/self-review loops — that exercises the distinct sub-skills an agent needs. It doubles as a “vibe coding” curriculum: the problems are small enough to reason about and hard enough to separate models.

Standing rules (how a score is earned)

- **Tools always ON.** No `--no-tools` runs. The point is the loop, not the model’s zero-shot recall. A two-layer path clamp keeps the model working inside its problem directory; verdicts still come only from the verifier.
- **Single-shot is canonical.** The headline score is one clean pass over all 30 problems, apples-to-apples with the rest of the fleet. A targeted rerun of only the misses is reported separately as “effective”, never promoted to the headline (re-rolling only the misses counts the upside but never re-checks whether a passing problem would drop on a second roll).
- **Zero-score rule.** Any lane at 0/ N is treated as a *server / harness / stack failure*, debugged as infrastructure, and marked `BROKEN-⟨cause⟩` — never recorded as a model capability. Every zero in this sweep was root-caused and fixed (see §6).
- **Attribute to provider $\times$ model**, never the model name alone.

- **Verdicts from exit codes.** Each problem ships an independent grader (`verify.py` / `check.py` / `pytest` / a scoring harness). The model is explicitly forbidden from editing the grader.

2 Executive summary

Twenty-four unique (model $\times$ backend) lanes completed the sweep. **Seven lanes reached a clean 30/30:** `oss120` (gpt-oss-120b, local dgx, vLLM 0.14.1), `kimi` (Kimi-K2.6, local dgx), `qwen36-27b` (chiatta00), `ornith-9b` (chiatta00, a reasoning model), `uic-qwen36-35b-a3b` and `uic-laguna-xs2` (uicgpu), plus `laguna-xs2` served directly on uicgpu (27/30 single-shot, effective 30/30 on a targeted rerun). `glm-5.2` (29/30, paid OpenRouter) and the newly-fixed `uic-ornith-9b` (28/30) lead the next tier.

Two findings stand out. First, **provider speed is a first-class axis:** fast local-dgx / nVIDIA-A100 / Intel-PVC backends finish 30/30 in minutes-to-hours, while spark-Ollama local lanes burn 7–8 hours for materially lower scores — throughput, not just accuracy, decides whether a lane is usable for real work. Second, **most zeros were infrastructure, not capability:** an Ollama chat-template gap silently dropped tool calls (`uic-ornith-9b` 0 $\rightarrow$ 28), a vLLM 0.13 streaming-flush bug (`oss120`), and a phantom un-resident model (`nemotron-super`) each masqueraded as a dumb model until the stack was fixed.

The problems that actually separated the fleet were the burst / concurrency / fan-out and numerical-stability tasks: **P24** (token-bucket rate limiter, 52% pass), **P4** (softmax numerical stability, 61%), **P17** (multi-backend codec round-trip, 61%), **P25** (3-stage fan-out pipeline, 61%), and **P13** (hand-rolled JSON parser, 70%). The easy problems (P5 matmul-correctness, P12 base-conversion, P15 word-count, P23 prime-count, P27 overlapping-substring) were passed by every lane that attempted them.

3 Integrated master table — all lanes, all backends

Every unique **model** $\times$ **backend** lane, deduped, in score order. The same model on two backends is two rows — serving path is a first-class variable. Score N/M with $M < 30$ = the lane did not cover all 30 problems (interrupted / partial).

#	Model	Provider / backend	Score	Time
1	<code>oss120</code> (gpt-oss-120b)	local dgx $\rightarrow$ vLLM 0.14.1	30/30	9m
2	<code>kimi</code> (Kimi-K2.6)	local dgx	30/30	3h18m
3	<code>qwen36-27b</code>	Intel PVC (1 tile) $\rightarrow$ agg	30/30	1h22m
4	<code>ornith-9b</code> (reasoning)	Intel PVC (1 tile) $\rightarrow$ agg	30/30	3h05m*
5	<code>uic-qwen36-35b-a3b</code>	nVIDIA A100 (MoE) $\rightarrow$ agg	30/30	52m
6	<code>uic-laguna-xs2</code>	nVIDIA A100 (MoE) $\rightarrow$ agg	30/30	49m
7	<code>laguna-xs2</code> (33B.A3B MoE)	nVIDIA A100 <code>ik_llama.cpp</code> :8080	27/30 (eff.30)	56m
8	<code>glm-5.2</code>	OpenRouter (paid)	29/30	44m
9	<code>gemma4-31b</code>	Intel PVC (1 tile) $\rightarrow$ agg	29/30	3h01m
10	<code>uic-ornith-9b</code> (reasoning)	nVIDIA A100 <code>llama.cpp</code> $\rightarrow$ agg	28/30	1h36m
11	<code>gemma4-12b</code>	Intel PVC (1 tile) $\rightarrow$ agg	28/30	3h15m
12	<code>devstral-small-2</code>	Intel PVC (1 tile) $\rightarrow$ agg	28/30	1h27m
13	<code>uic-gemma4-26b-q4</code>	uicgpu (Q4) $\rightarrow$ agg	26/30	1h12m
14	<code>nemotron-3-super 120B</code>	OpenRouter free	26/30	1h48m
15	<code>uic-gemma4-26b-q8</code>	uicgpu (Q8) $\rightarrow$ agg	24/30	1h40m
16	<code>devstral2-24b</code>	spark (ollama)	24/30	2h14m
17	<code>uic-ornith-35b</code>	nVIDIA A100 $\rightarrow$ agg	23/30	1h29m
18	<code>nemotron-3-ultra 550B</code> (NVFP4)	local dgx (rbh101)	22/26	hung@P27

#	Model	Provider / backend	Score	Time
19	qwen3-14b	spark (ollama)	20/28	4h41m
20	gemma4-e4b	Intel PVC (1 tile) → agg	20/30	1h39m
21	gemma4-e2b	Intel PVC (1 tile) → agg	20/30	1h02m
22	nemotron-3-nano (Nemotron-H MoE)	30B spark (ollama)	19/30	7h36m
23	llama70	local dgx	18/29	7h16m
24	nemotron3 33B (Nemotron-H Omni)	spark (ollama)	16/30	7h31m

* ornith-9b time excludes the ~6.5 h P30 harness hang (a bug, not compute). Total time = active START→DONE wall-clock, summed across resume segments.

Backend legend.

- **Intel PVC** — an Intel Data Center GPU Max 1550 cluster; each *dense* model runs on a single 64 GB PVC tile, fronted by a local aggregator.
- **nVIDIA A100** — an 8×A100-80GB DGX; models run on one or a few GPUs, typically MoE architectures (laguna, qwen36-a3b, ornith).
- **local dgx** — locally-served DGX endpoints (the former “CELS” / *chicago-N* lanes, incl. the rbh101 host for the 550B); swept directly, never through LiteLLM.
- **spark (ollama)** — spark nodes serving via Ollama.
- **OpenRouter** — hosted API (free tier preferred; paid tagged).

4 The 30 problems — definitions, what each measures, how the fleet did

The problems fall into five families. Pass rates below are over the 23 lanes that recorded per-problem verdicts (partial lanes counted only where they reached the problem).

4.1 Family A — Write-from-spec (P1, P2, P7, P17, P25)

The model is handed a specification and an empty directory; it must produce working code and run the verifier itself. This tests spec-following, multi-file scaffolding, and (for P7/P17/P25) importing across a directory tree.

- **P1 — stats CLI (91%).** Write *cli.py* reading numbers from *STDIN*, print *count/min/max/mean/median/stdev* (sample *stdev*, $n-1$; even-count *median* = *avg of two middle*). *Stdlib only*. Measures exact output-format discipline and the sample-vs-population *stdev* trap. Missed only by the two smallest gemma lanes.
- **P2 — LRU cache (96%).** Implement *LRUCache* with $O(1)$ *get/put*, correct *eviction*, AND write + run *pytest* tests. Tests whether the model writes its own tests before declaring done. Only nemotron3-33b missed.
- **P7 — dispatch router (74%).** Given *work/{csv,kv,json} /backend.py* each exposing *parse*, write *dispatch.py* routing by *kind*. First cross-directory import test. Small models and the Nemotron-H lanes stumble on the import path.
- **P17 — codec round-trip (61%, hard).** Given three backends each exposing *dumps/loads*, write *codec.py* doing *round_trip(fmt,obj)* via the matching backend. P7 with two functions and a round-trip invariant — one of the four hardest. Nine lanes failed, all of them the smaller / slower

models: the difficulty is holding a two-function contract across three import targets without editing the verifier.

- **P25 — 3-stage fan-out pipeline (61%, hard).** *Given `work/{clean,agg,fmt}/stage.py`, write `pipeline.py` exposing `run(rows)=fmt(agg(clean(rows)))`.* A composition / fan-out task; notably the *only* problem `g1m-5.2` missed, and one the otherwise-strong laguna-direct lane missed on its single shot. Fan-out composition is a genuine variance point even for top models.

4.2 Family B — Fix failing tests (P3, P11, P13, P20, P22, P29)

A file with a bug and a red pytest/diagnostic; the model must make the smallest fix and *is forbidden from editing the test*. Tests debugging-from-evidence and restraint.

- **P3 — roman numerals (91%).** Subtractive notation (IV, IX, ...). The classic “smallest fix” discipline test.
- **P11 — balanced parens (91%).** Stack-based `()[]{}` matching, ignoring other chars.
- **P13 — mini JSON parser (70%, hard).** *Parse a JSON subset (`null/bool/int/float/string/array/object`, nesting, whitespace) WITHOUT `json` or `eval`.* The hardest fix-family problem — a real recursive-descent parser. Seven lanes failed, cleanly separating the mid-tier from the top.
- **P20 — TTL cache (91%).** LRU eviction *and* TTL expiry via an injected `clock()`. Tests dependency-injection of time.
- **P22 — topological sort (96%).** Valid topo order, raise on cycle. Only llama70 missed.
- **P29 — expression calculator (87%).** `+ - * /` with precedence and parentheses, no `eval` (shunting-yard / recursive descent).

4.3 Family C — Numerical / algorithmic fix (P4, P8, P12, P14, P18, P21, P23, P26, P28)

Fix-a-bug tasks where the bug is mathematical: overflow, cancellation, wrong recurrence, or an inefficient algorithm. Tests numerical literacy.

- **P4 — softmax stability (61%, hard).** *`fast.py` softmax overflows; fix by subtracting the max before exp.* The single most *diagnostic* numerical problem: nine lanes failed, including otherwise-strong ones (gemma4-31b, both ornith uic lanes). Knowing the “subtract max” trick is a sharp capability signal.
- **P8 — ODE integrator (83%).** Use RK4 and compute $t = t_0 + \text{step} \cdot h$ (never accumulate), and append a root-cause lesson to the agent’s `memory.md` — so it also tests memory-write behaviour.
- **P12 — base conversion (100%).** Round-trip base 2–36, 0 and negatives. Everyone passed.
- **P14 — Dijkstra (87%).** Shortest path with a heap, correct unreachable handling.
- **P18 — Welford variance (91%).** Online sample variance, numerically stable, constant-stream $\rightarrow 0$; also writes a memory lesson.
- **P21 — interval merge (91%).** Merge overlapping/touching intervals.
- **P23 — prime count (100%).** Correct for small n . Everyone passed.
- **P26 — Newton square root (87%).** Full double precision via Newton’s method, no `math.sqrt`; writes a memory lesson.
- **P28 — matrix determinant (96%).** $N \times N$ determinant matching numpy (LU / cofactor). Only llama70 missed.

4.4 Family D — Optimize to a target (P5, P15)

Correct-but-slow code; the model must make it faster while staying correct, iterating against a measured throughput target. Tests profile-driven optimization inside a budget.

- **P5 — matmul GFLOP/s (100%).** Fix correctness first, then vectorize (`np.dot / @`) to hit a GFLOP/s target. Everyone passed.
- **P15 — word-count Mtok/s (100%).** Correct top- k , then speed up with `Counter / heapq.nlargest`. Everyone passed.

That both optimization problems were universally passed says the fleet is comfortable with the “make one change, re-measure” loop when the target is a throughput number.

4.5 Family E — Tournament & critic loops (P6, P9, P10, P16, P19, P24, P27, P30)

The most agentic family. Best-of- N tournaments spawn several candidate solutions in parallel and score them; critic problems require the model to review its own code (write a `CRITIQUE.md`, then fix). Tests parallel orchestration and self-review.

- **P6 — deterministic reduce + critic (83%).** Make `parallel_sum` bit-identical regardless of chunk count (`math.fsum / fixed ordering`), with a hostile-reviewer critic pass.
- **P9 — Levenshtein tournament (91%).** Best-of-4 candidates, scored on correctness + speed (two-row DP). Missed only by the two uic-ornith lanes.
- **P10 — prime-sieve tournament + critic (83%).** Best-of-4 sieve to 2M, plus a critic that may fix and write a `VERDICT`.
- **P16 — thread-safe counter + critic (87%).** Add a lock around a shared counter, with a race-condition critic pass.
- **P19 — fast-doubling Fibonacci tournament (91%).** Best-of-4, exact bigint, $O(\log n)$ at $n = 200,000$.
- **P24 — token-bucket limiter + critic (52%, HARDEST).** *Correct burst-then-refill with an injected clock, plus a hostile critic on refill math.* The single hardest problem in the suite: eleven of 23 lanes failed. It combines three traps — burst-must-not-exceed-capacity, correct fractional refill, and clock injection — and adds a self-review loop on top. This is the sharpest separator between the top tier and everyone else.
- **P27 — overlapping-substring tournament (100%).** Count overlapping occurrences (`'aa'` in `'aaaa' = 3`). Everyone passed.
- **P30 — sort-kernel capstone (91%).** Best-of-4 sort + critic + bench; the finale. Notably the problem whose grader had a `SCORE_TIMEOUT` bug that caused a ~6.5h harness hang and false failures until fixed (see §6) — a reminder that a “model failure” on a capstone is often a harness failure.

5 What separated the fleet

- **The top tier is real and tight.** Seven lanes at 30/30 span four backends (local dgx, nVIDIA A100, Intel PVC) and four model families (gpt-oss, Kimi, Qwen-3.6, ornith, laguna). A clean 30/30 is achievable at 9B (ornith-9b) — size is not the gate; *tool-use training and a correct serving stack* are.
- **Concurrency + injected-clock is the wall.** The three lowest pass rates after the codec/fan-out tasks are all timing/concurrency: P24 token-bucket (52%), P20 TTL, P16 thread-safe counter. Models that can write a sort or a Dijkstra routinely mishandle “refill r tokens/sec using `clock()`”.
- **Numerical stability is a clean IQ test.** P4 (softmax) at 61% failed by half the mid-tier despite being a one-line fix — it tests whether the model *knows* the overflow pattern, not whether it can code.

- **Fan-out composition has irreducible variance.** P25 was missed even by glm-5.2 and single-shot laguna. Parallel-backend / multi-stage orchestration (Family E + P17/P25) is exactly where top models occasionally slip, which is why the canonical score is single-shot and 30/30 is genuinely hard to hit cold.
- **Throughput sorts the tail.** Every lane below 24/30 except the partials is a slow spark-Ollama local model burning 4–8 h. For real agentic work the effective usability cliff is not just the score — an 18/30 lane that takes 7 h is not a candidate for a Kanban worker regardless.

6 Infrastructure failures caught by the zero-score rule

Every 0/*N* in this sweep was an infrastructure bug, not a model. Per the standing rule they were debugged as stack failures and fixed:

- **uic-ornith-9b:** 0/30 → 28/30. Ollama GGUF blobs carry no chat template, so `--jinja` silently emitted no tool calls — the loop never acted. Fixed by serving through `llama.cpp` with an explicit `--chat-template-file`; the real score is 28/30.
- **oss120 streaming-flush (vLLM 0.13).** A streaming-flush bug in vLLM 0.13 truncated tool-call deltas; upgrading the local-dgx endpoint to vLLM 0.14.1 restored a clean 30/30.
- **nemotron-super phantom model.** The 120B was never resident on the host (weights > RAM) — every call failed. Re-routed to a real endpoint for the 26/30 result.
- **P30 SCORE_TIMEOUT hang.** A grader-side timeout bug caused a ~6.5 h harness hang and false P30 failures on ornith-9b and uic-qwen36; after the fix both reached 30/30. The hang time is excluded from wall-clock (it was a bug, not compute).
- **PYTHONPATH conda leak.** A conda PYTHONPATH export leaked into every pi bash-tool subshell and crashed pytest-based problems; scrubbed at the top of the harness.

The lesson the rule encodes: **a literal zero on a tool-use benchmark is almost always the stack, not the model.** Recording it as capability would have libeled five perfectly capable models.

7 Model-identity & provenance notes

- **Nemotron family.** `nemotron3 33B` is *Nemotron-H 33B “Omni”* (`arch nemotron_h_omni`, Mamba-hybrid, vision+tools) — it wastes capacity on an unused vision stack and scores 16/30. `nemotron-3-nano 30B` is *Nemotron-H 30B “MoE”* (`nemotron_h_moe`, text-only, 1.05M ctx) and beats its Omni sibling at 19/30. `nemotron-3-ultra 550B` (A55B, NVFP4 on local dgx, rbh101 host) is a distinct dense-MoE and hung at P27 (22/26 partial). All three verified against live `ollama show` / the local endpoint map, not from tags.
- **glm-5.2** is paid OpenRouter (`z-ai/glm-5.2`, no free tier); tagged as paid. It replaces the retired `glm-4.7-flash` (1/28 capability ceiling) — a full-generation leap.
- **Sanctioned providers:** OpenRouter (free tier preferred; paid allowed when tagged), Argo/ALCF, and locally-served DGX endpoints. Local-dgx sweeps run direct, never through LiteLLM. Scores are attributed per provider × model.
- **Broken runs preserved.** All `BROKEN-*` run directories are archived with self-documenting names, never deleted.

Source of truth: `~/pi-problems-30/FLEET_BENCHMARK_MASTER.md` (v7). Harness: `run_model_30.sh`. Problem definitions: `p1/-p30/` + embedded task prompts. All artifacts synced to `~/Dropbox/XFER/pi-fleet-benchmark/`.